Computer Networks I
Reti di Calcolatori I

The Transport Layer: TCP

Lecture 13

UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DIETI

UNINA

Transport Layer

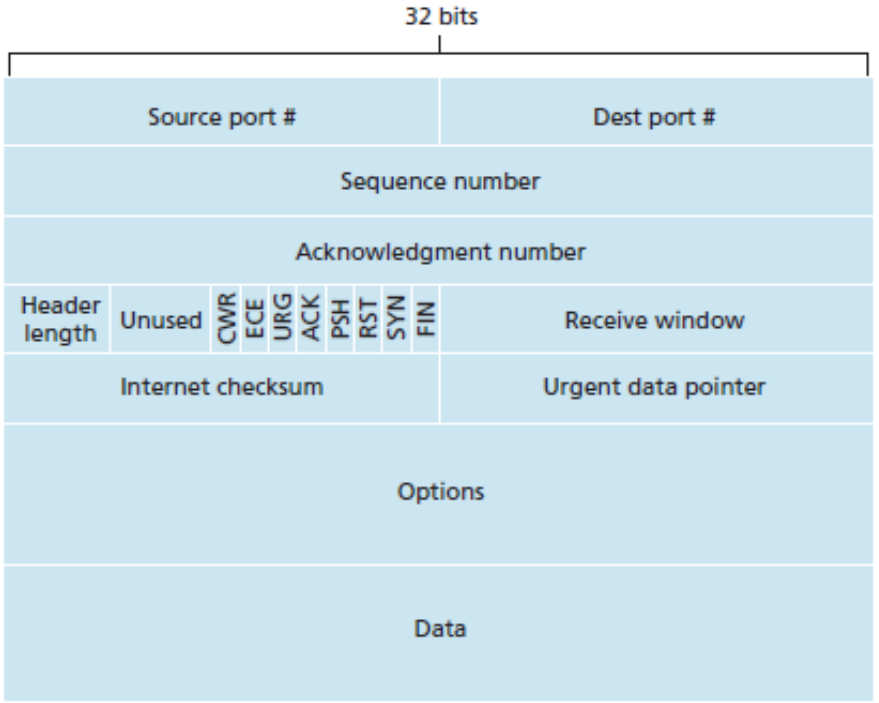
[Remind] Responsibilities of Transport Layer

- Transport-layer protocols (UDP and TCP) have four main responsibilities:
  1. **Process-to-process delivery:** messages are delivered independently of where these processes are.
     - Note that this is **different from host-to-host delivery** (i.e., between two different machines), which is responsibility of the lower-level IP protocol.
  2. **Integrity checking:** by including error detection fields into the segments' headers.
  3. **Reliable data transfer**: ensuring that data is delivered from sending process to receiving process, correctly and in order.
  4. **Congestion/flow control**: it prevents connection from swamping network devices (links or routers) with an excessive amount of traffic. This service is useful to improve performance and **is very beneficial to the whole (internet) network**.

- In particular, UDP (faster but simpler) provides **only the first two services**, while TCP provides all the four ones.

Transport Layer
[Recall] TCP Segment

- The TCP segment consists of **header fields** and a **data field**, the latter contains some application data **of at most MSS size**.
- The **header** contains several fields:
  - **Sequence number** (32-bit) for reliable data transfer.
  - **Acknowledgment number** (32-bit) for reliable data transfer.
  - **Receive window** (16-bit) used to indicate the number of bytes that a receiver is willing to accept (for flow control).
  - **Header length** (4-bit) specifies the number of 32-bit words contained into the header. Note that **TCP header is variable due to the options** field.
  - **Options field** (K-bit) used for optional processes such as negotiating the MSS, time-stamping, etc.
  - **Flags** (6/12-bit) including:
    - ACK bit indicates that **this segment is an ACK** packet (so the acknowledgment number field is in use).
    - RST, SYN, and FIN bits used for **connection setup and teardown**.
    - CWR and ECE bits used in explicit congestion notification (optional).
    - PSH bit tells the receiver to pass the data to the application immediately.
    - URG bit indicates that the segment contains "urgent" data.
  - **Checksum** (16-bit): for integrity check.
  - **Urgent data pointer field** (16-bit) indicates the location of the last byte of the urgent part of the data.



Transport Layer

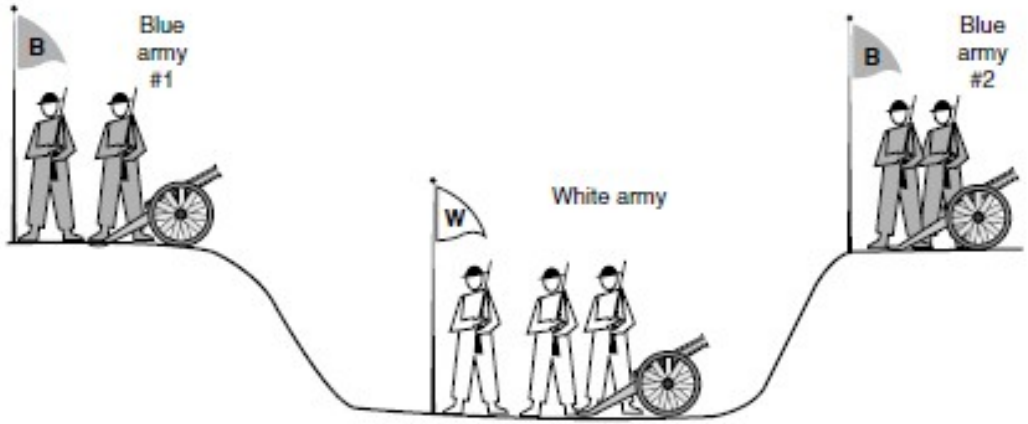
TCP Problems in Connection Management

- Since TCP is connection-oriented **two hosts must agree** during both the **opening** and the **closing** procedures.
- Knowing that messages could be **lost or damaged** on the network, **it is quite difficult** for two hosts to find such agreement.
- If messages are lost during transmission **one end of the communication can be open or closed while the other is not**.
- To avoid (or better to mitigate) this issue TCP implements a procedure called **three-way handshake** in which open/close connection requests **have to be acknowledged** by hosts before a connection can be established/released.

Transport Layer

TCP Problems in Connection Management

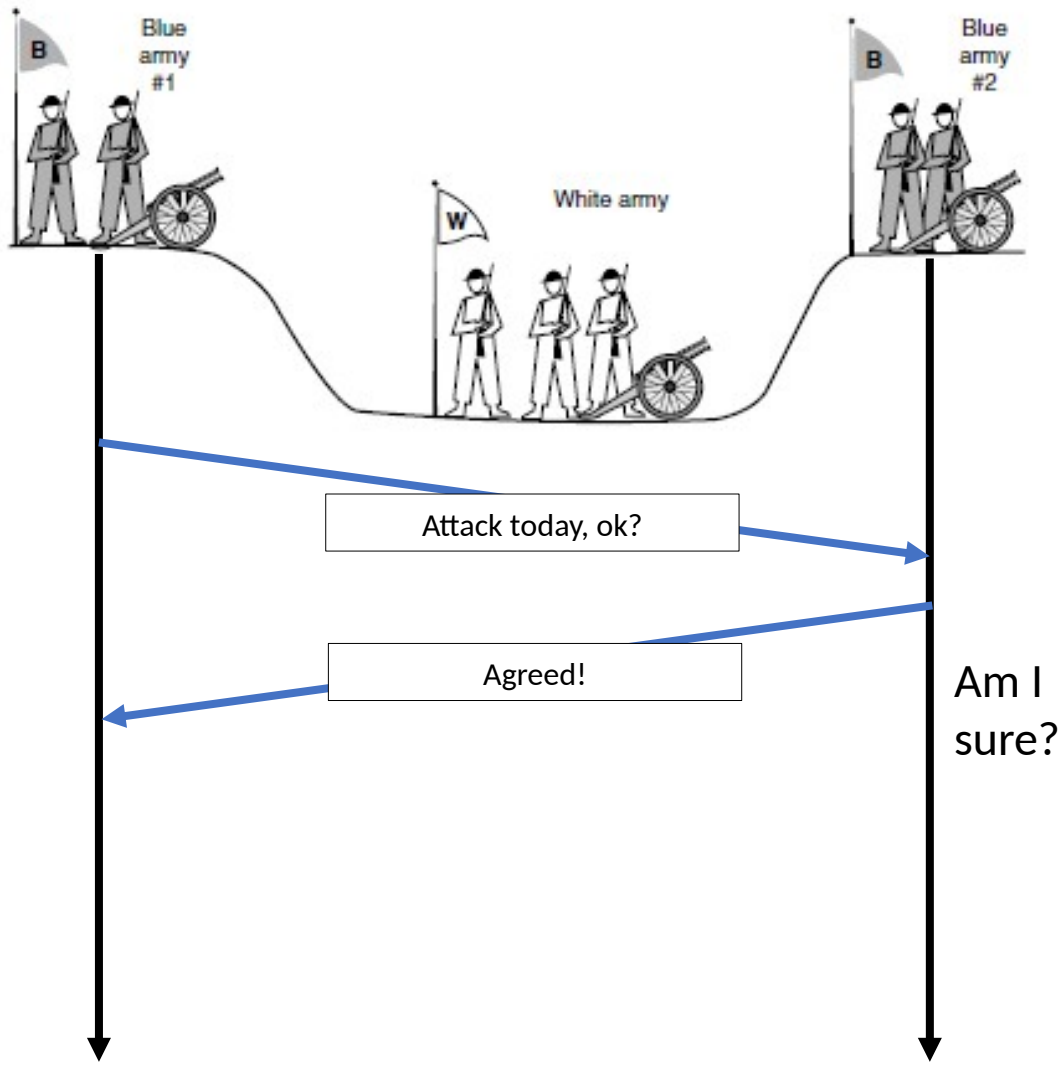
- The two-army problem:
  - Imagine a **white army encamped in a valley** and, on both hillsides, there **are 2 enemy blue armies**.
  - The **white army is larger than either of the blue armies** alone, but together the blue armies are larger, they will be victorious only if the attack is simultaneous.
  - To synchronize their attacks, **blue armies must send messengers through the valley** where they might be captured (unreliable communication).
  - Does a protocol exist that allows the blue armies to win?



Transport Layer

TCP Problems in Connection Management

- Suppose that the commander of **blue army #1** sends a message reading: ‘‘I propose we attack today, is it ok?’’
- Now suppose that **the message arrives**, the commander of blue army #2 agrees, and his **reply gets safely back** to blue army #1.
- Will the attack happen? Probably not, because **commander #2 does not know if his reply got through**. If it did not, blue army #1 will not attack.

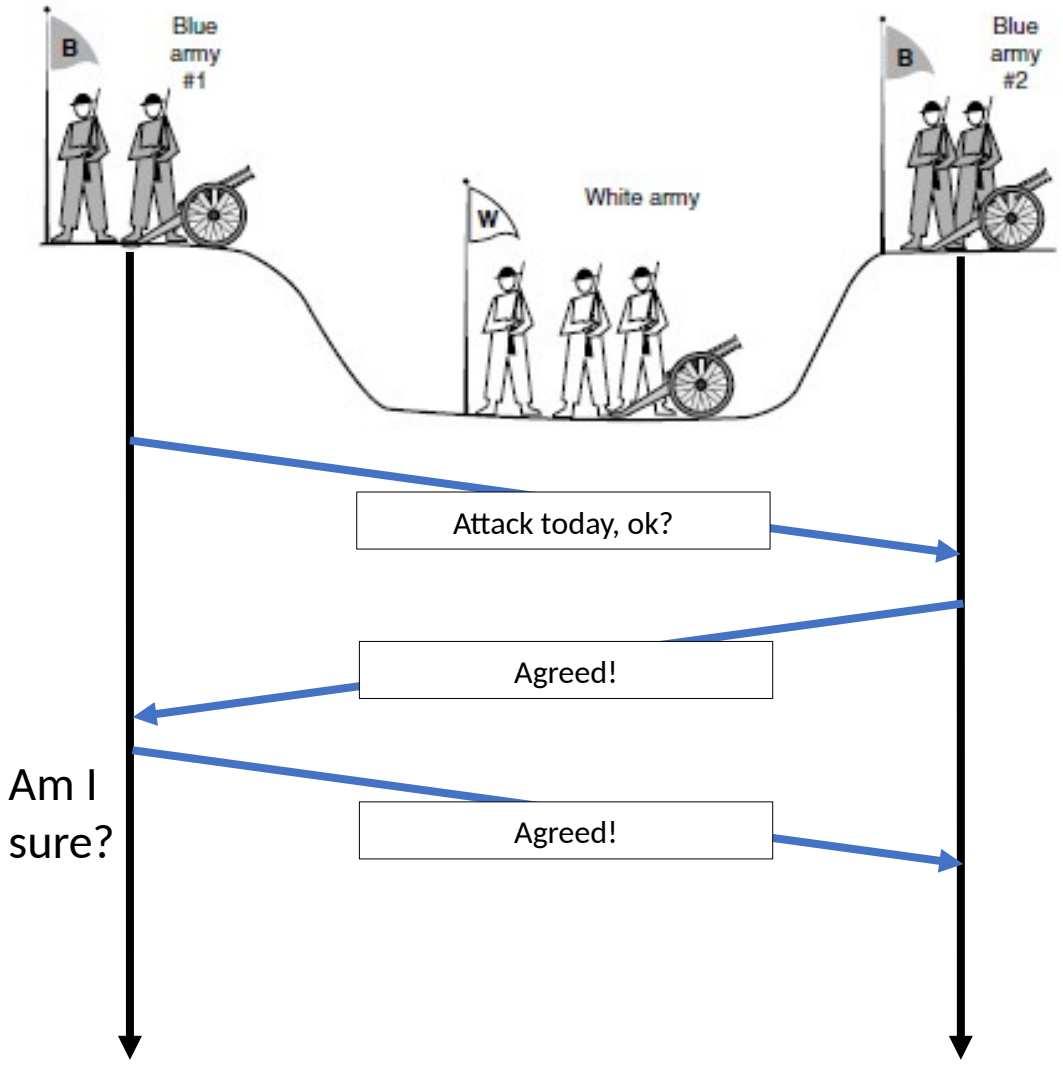


Transport Layer
TCP Problems in Connection Management

- Let's make it a **three-way handshake**: the first commander (of the original proposal) **must acknowledge** the response.

- Assuming **no messages are lost**, blue army #2 will get the acknowledgement, but the commander of blue army #1 will now hesitate (he does not know if his acknowledgement got through).

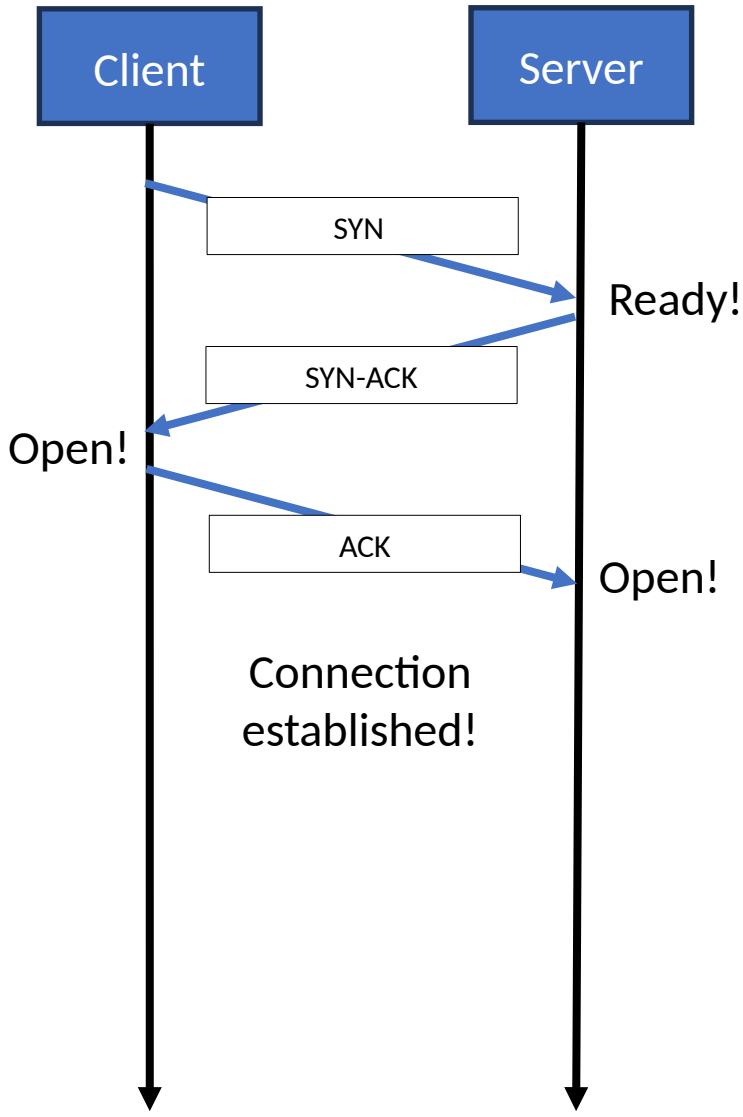
- Now **we could make it a four-way handshake**, but that does not help either. In fact, **it can be proven that no protocol exists that works**.
  - **Three-way handshake** is not perfect, but it is usually adequate!



Transport Layer
TCP Connection Establishment

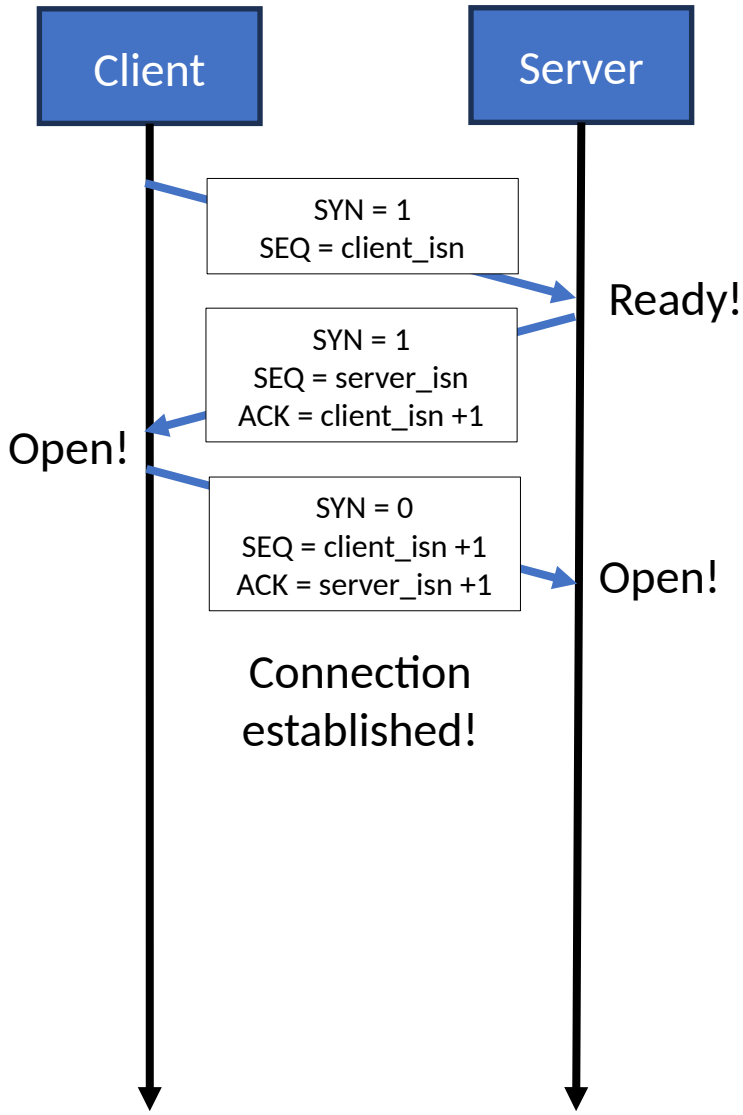
- In TCP **three-way handshake** is performed to establish connection:
  - Client sends a **connection request** (SYN segment) to the server.
  - Server responds with a **special acknowledgment** (SYN-ACK segment).
  - Client sends back a **final acknowledgment** (ACK segment).

- TCP connection establishment is a **delicate procedure** that can also add significantly delays.
  - There is typically a **timeout** (30-60 secs) to complete the handshake, after that **the procedure is aborted**.
  - Some **attacks** (e.g., SYN flood) happen here.



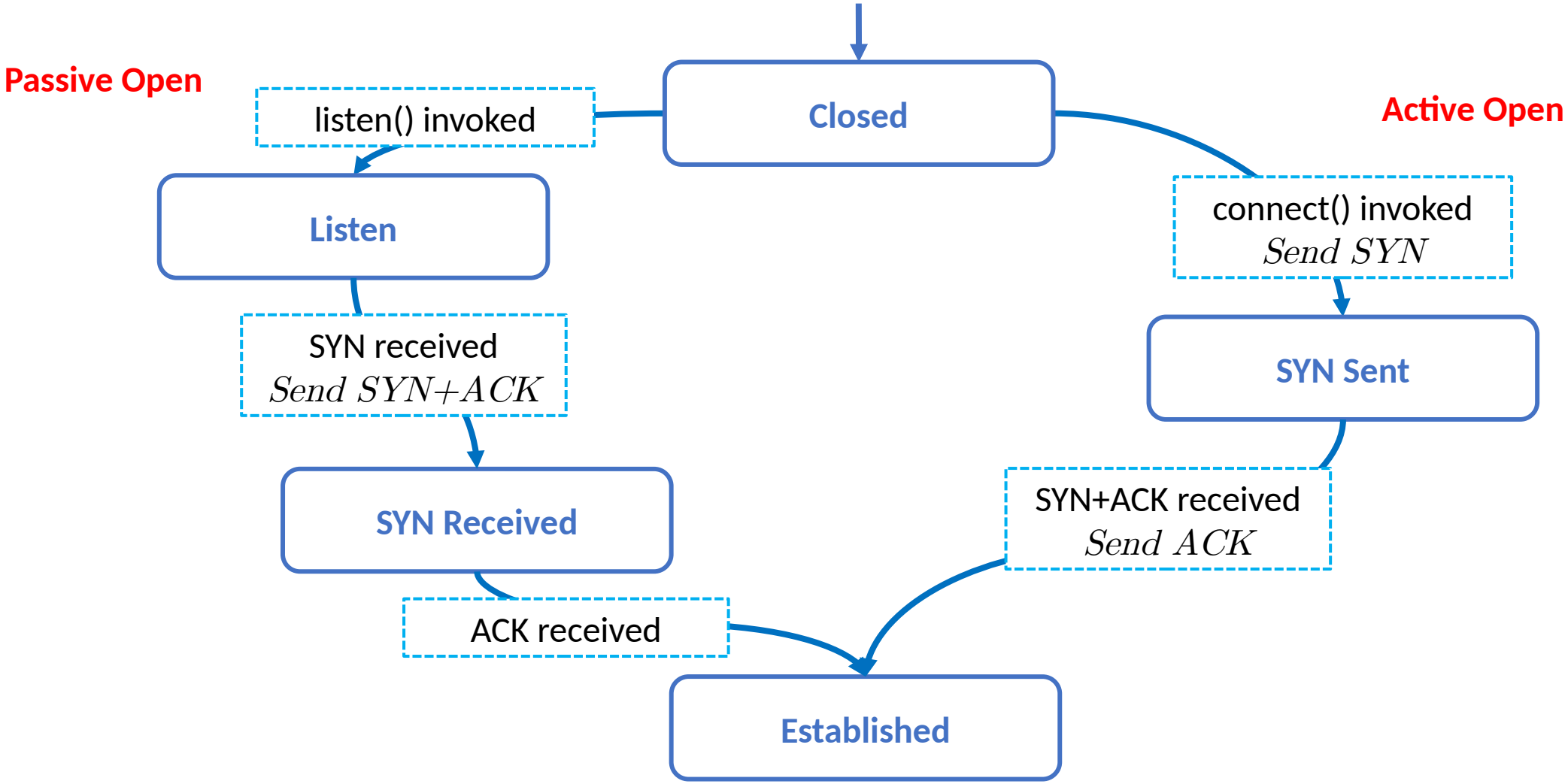
Transport Layer
TCP Connection Establishment

- Details of the **three-way handshake** procedure:
  1. The client sends a **SYN segment** having:
     - No application-layer data (payload).
     - The SYN bit set to 1.
     - A random initial sequence number (client_isn) as sequence number.
  2. When the above SYN segment (hopefully) arrives, the **server allocates TCP buffers and variables** and sends back a **SYNACK segment** having:
     - No application-layer data (payload).
     - The SYN and ACK bits set to 1.
     - The acknowledgment number set to client_isn+1.
     - A random initial sequence number (server_isn) as sequence number.
  3. When the SYNACK segment (hopefully) arrives, the **client also allocates buffers and variables** to the connection and sends to the server a final **ACK segment** having:
     - Possibly application-layer data.
     - The SYN bit set to 0 (connection is established) and ACK bit set to 1.
     - The acknowledgment number set to server_isn+1.



Transport Layer
TCP Connection Establishment

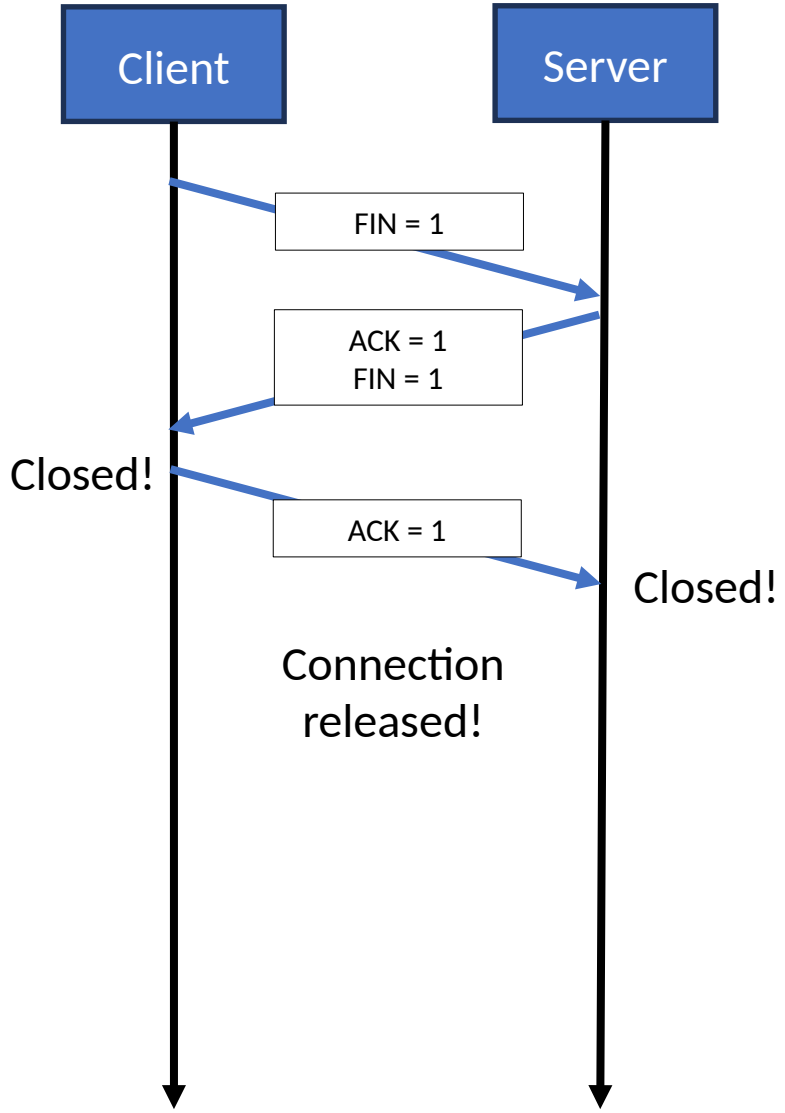
- Simplified state diagram for TCP connection establishment.



Transport Layer
TCP Connection Release

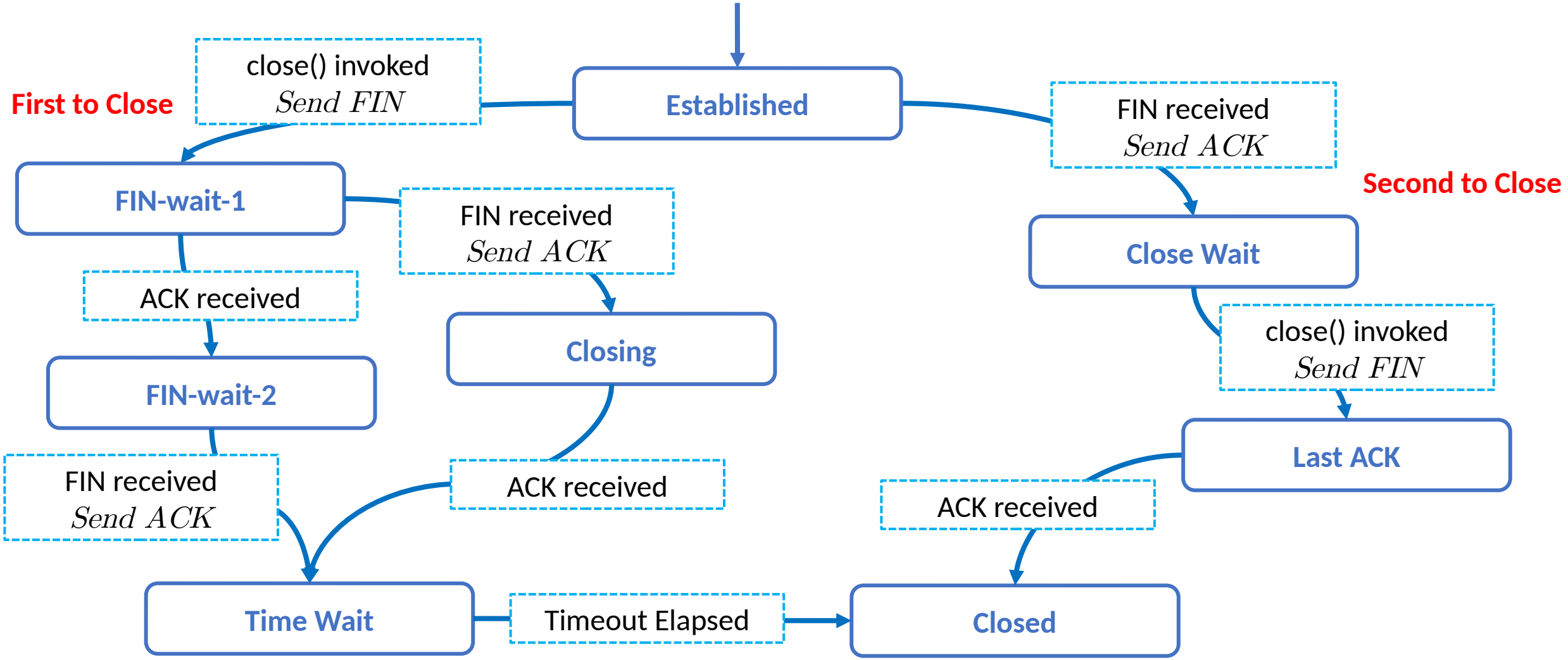
- TCP connection release (aka **teardown**) can be performed by either host.

- Let's assume client is closing the connection, the **three-way handshake** is performed as follow:
  1. The client sends a **special shutdown segment** (FIN segment) to the server having the FIN bit set to 1.
  2. When the server receives this segment, it **sends back an acknowledgment/shutdown segment**, having ACK to 1 and FIN bit set to 1.
     - Note: ACK and FIN can be sent in the same segment or in two separated ones.
  3. Finally, the client **acknowledges** the server's shutdown segment.



Transport Layer
TCP Connection Release

- Simplified state diagram for TCP connection release.



Transport Layer
TCP Connection Release

- How do we save ourselves from packet loss?
- Since three-way handshake is not perfect, TCP rely on timers to eventually close connections or to send again requests. Here some examples during connection release: